

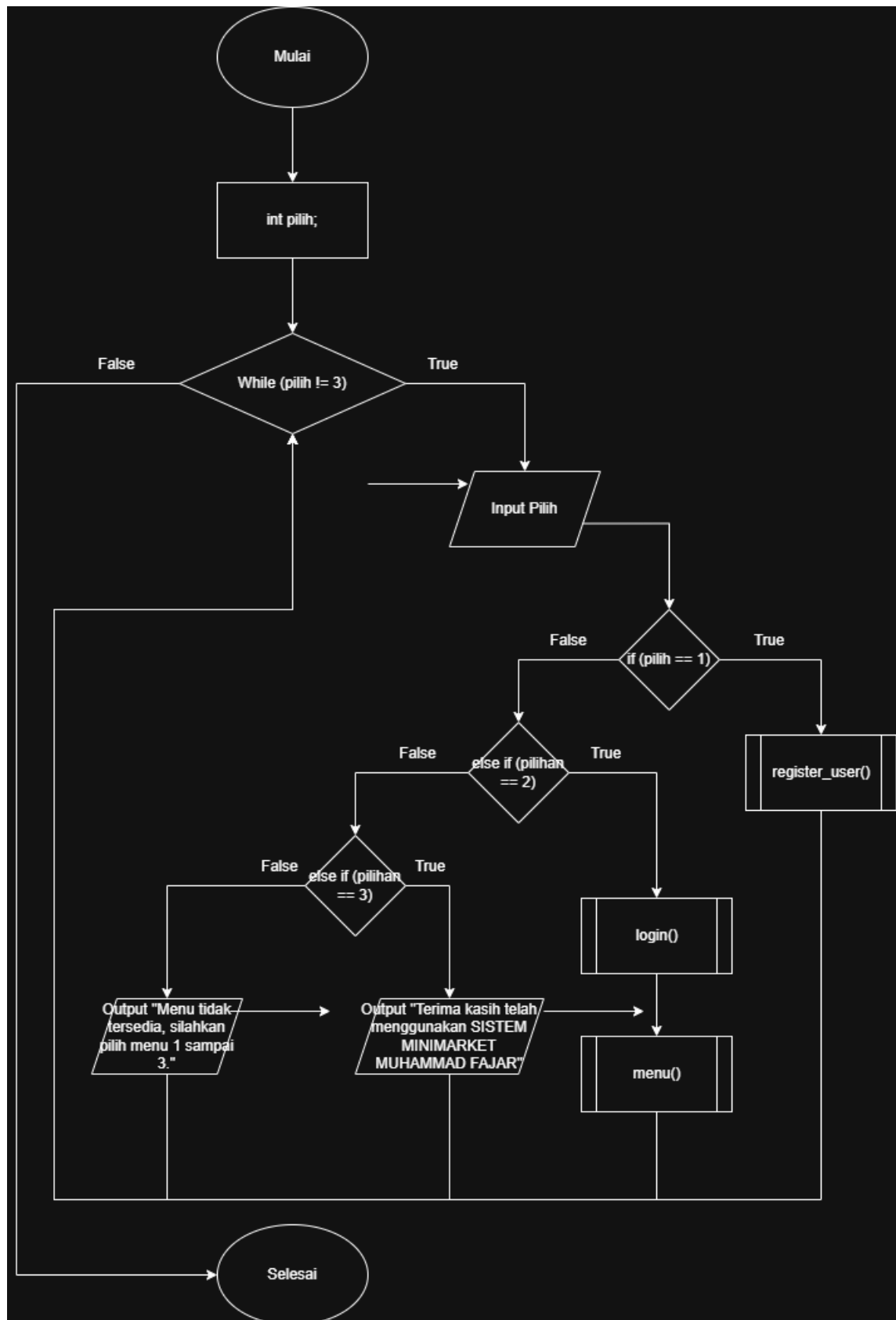
**LAPORAN PRAKTIKUM
POSTTEST 2
ALGORITMA PEMROGRAMAN LANJUT**



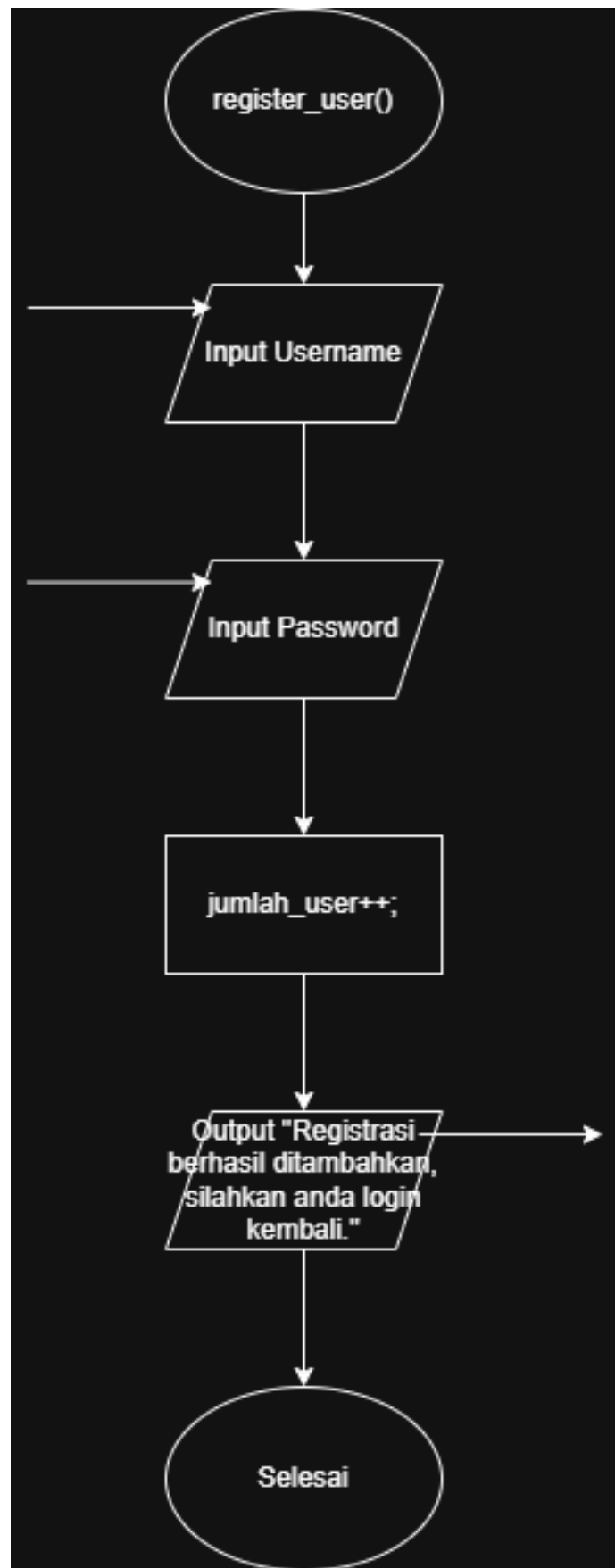
**Disusun oleh:
Muhammad Fajar (2509106117)
Kelas (C2 '25)**

**PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026**

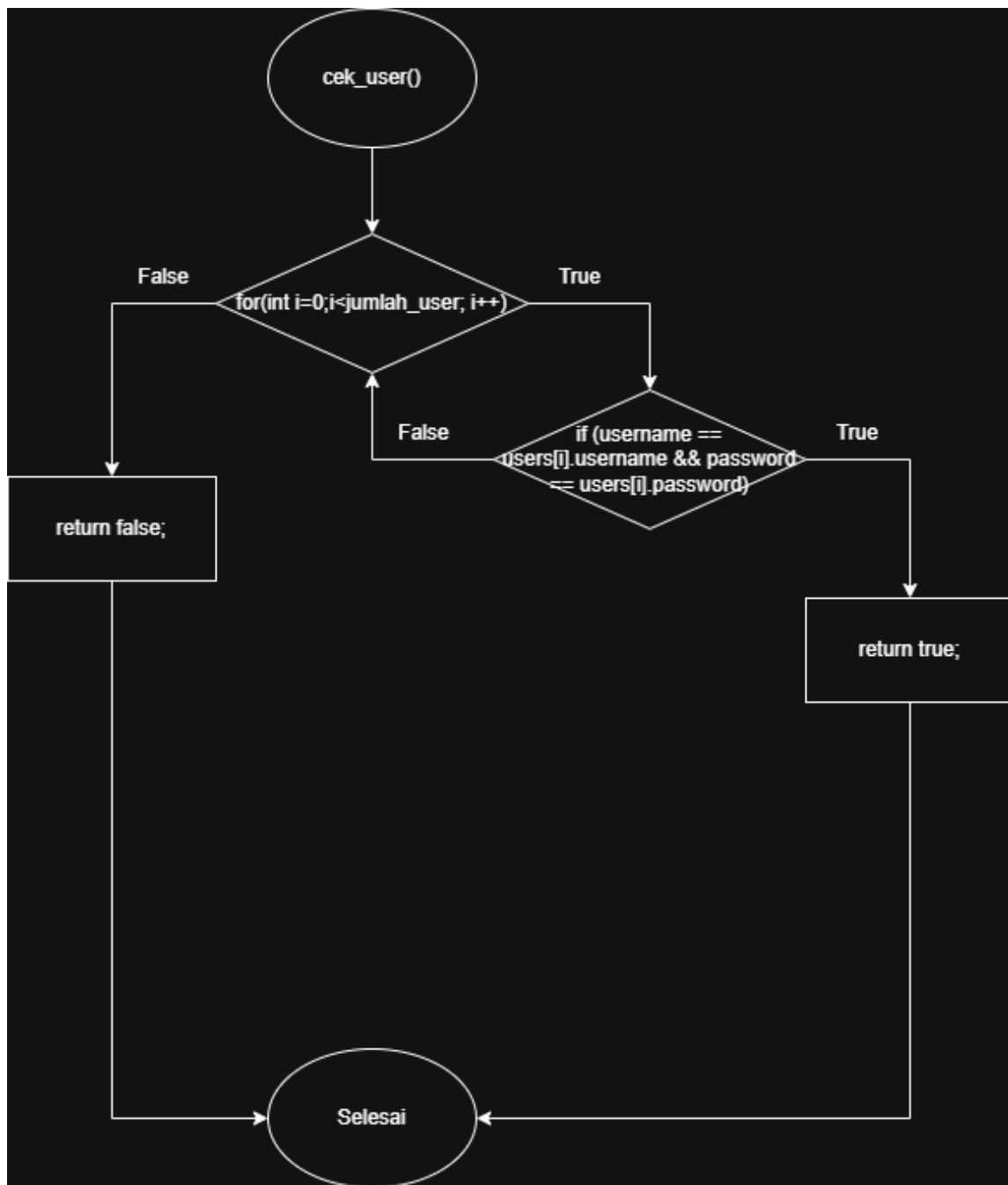
1. Flowchart



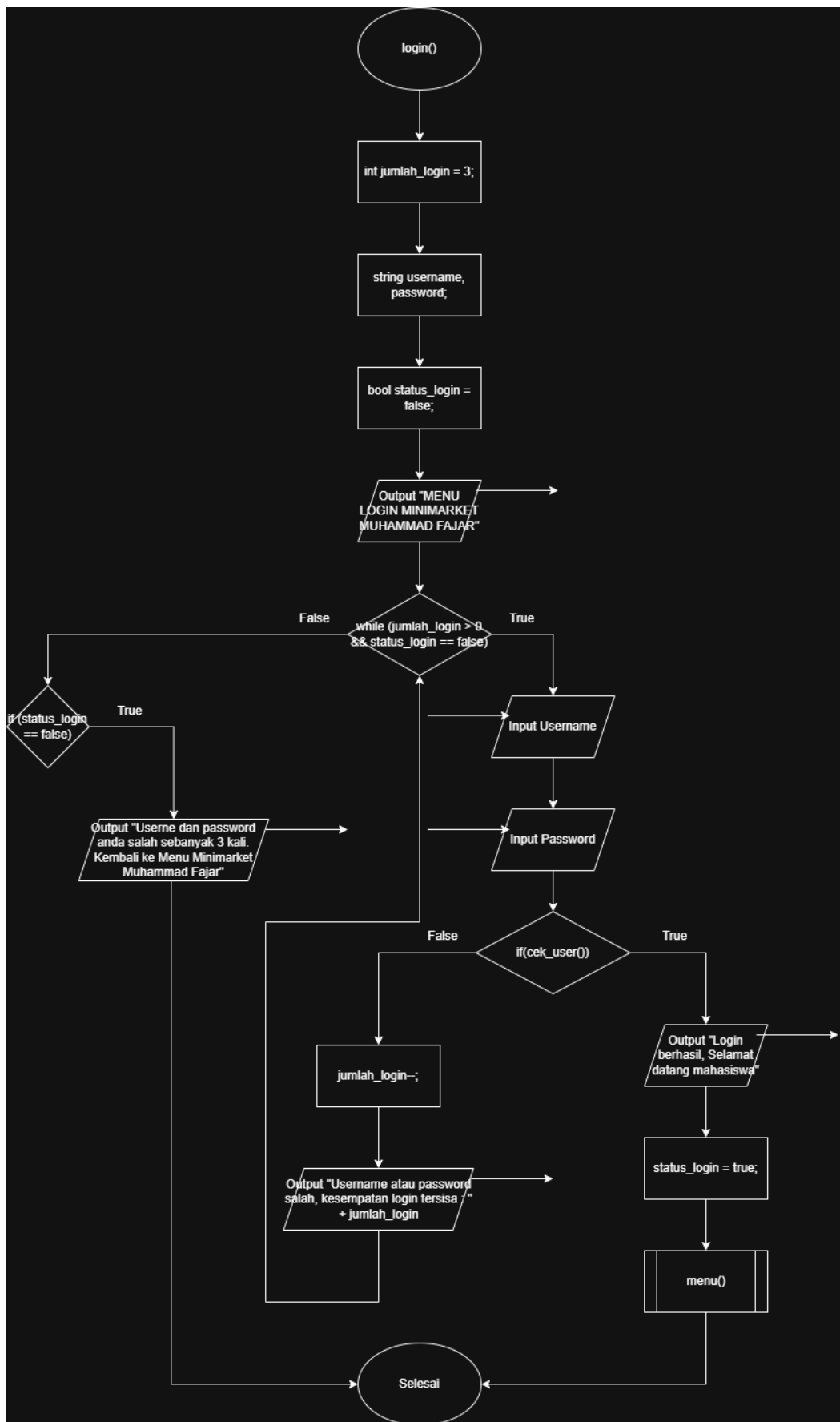
Gambar 1.1 Flowchart



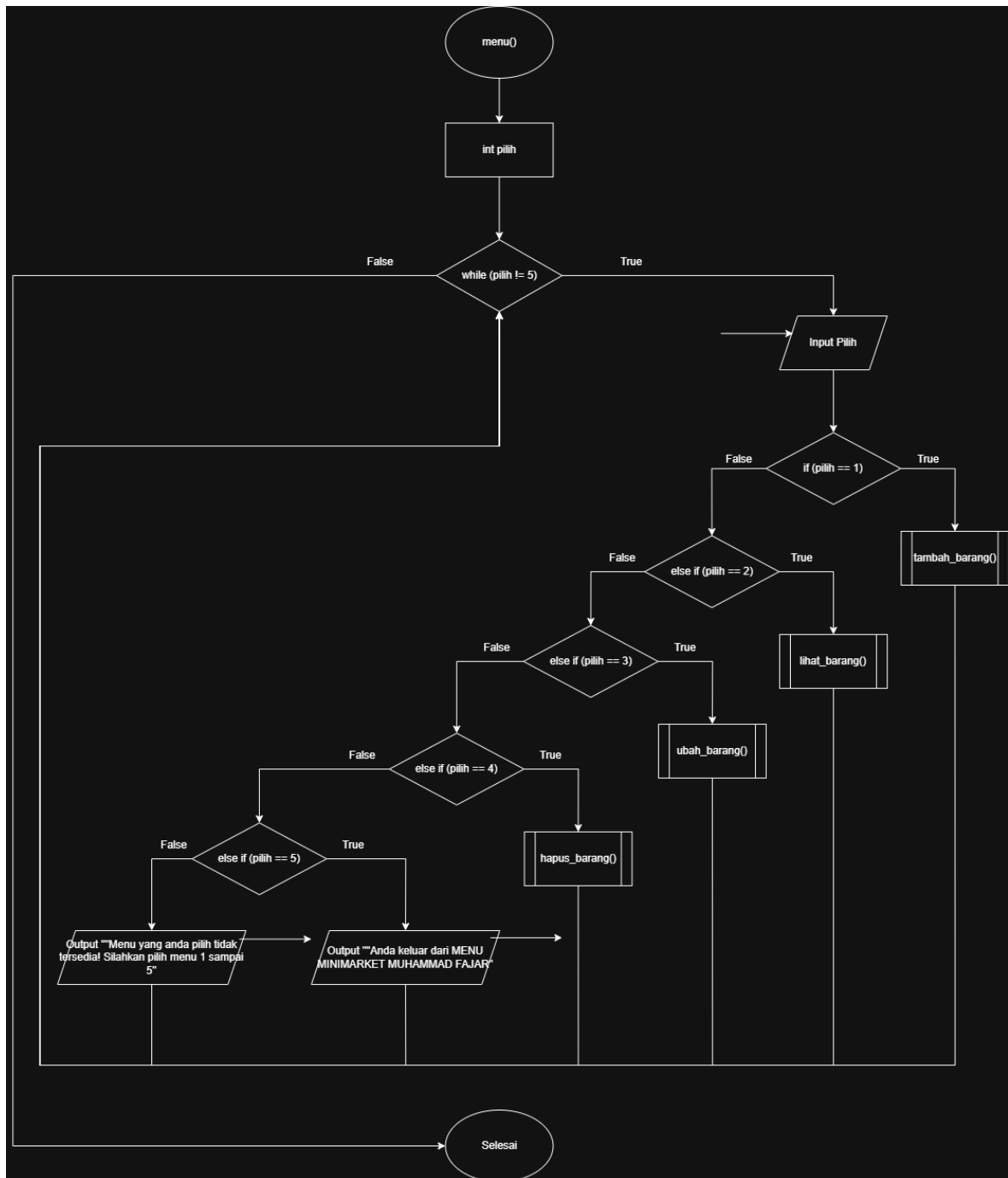
Gambar 1.2 Flowchart



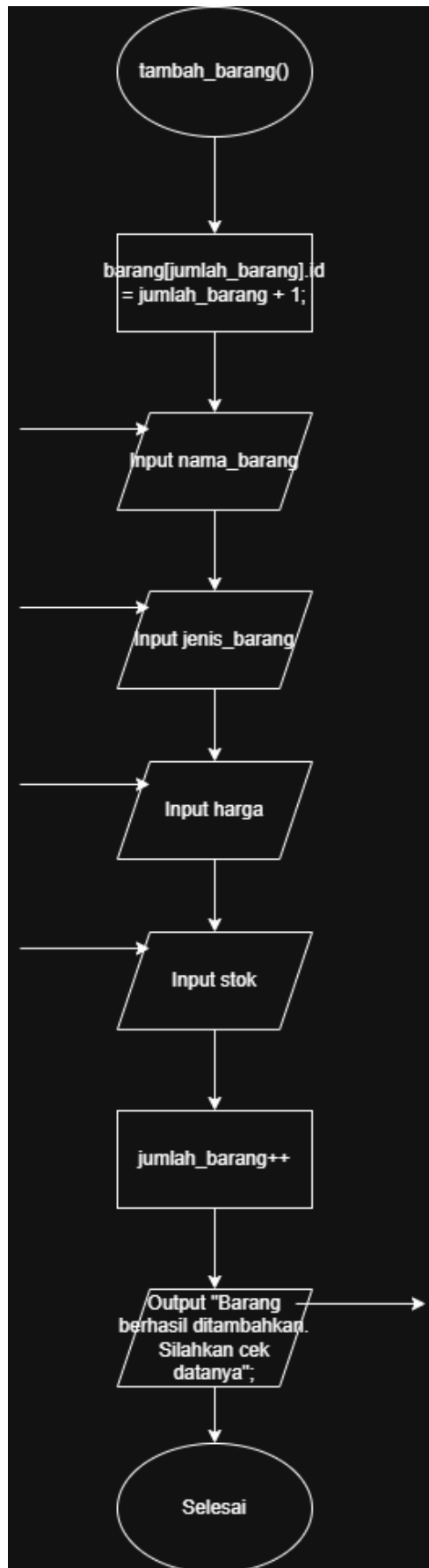
Gambar 1.3 Flowchart



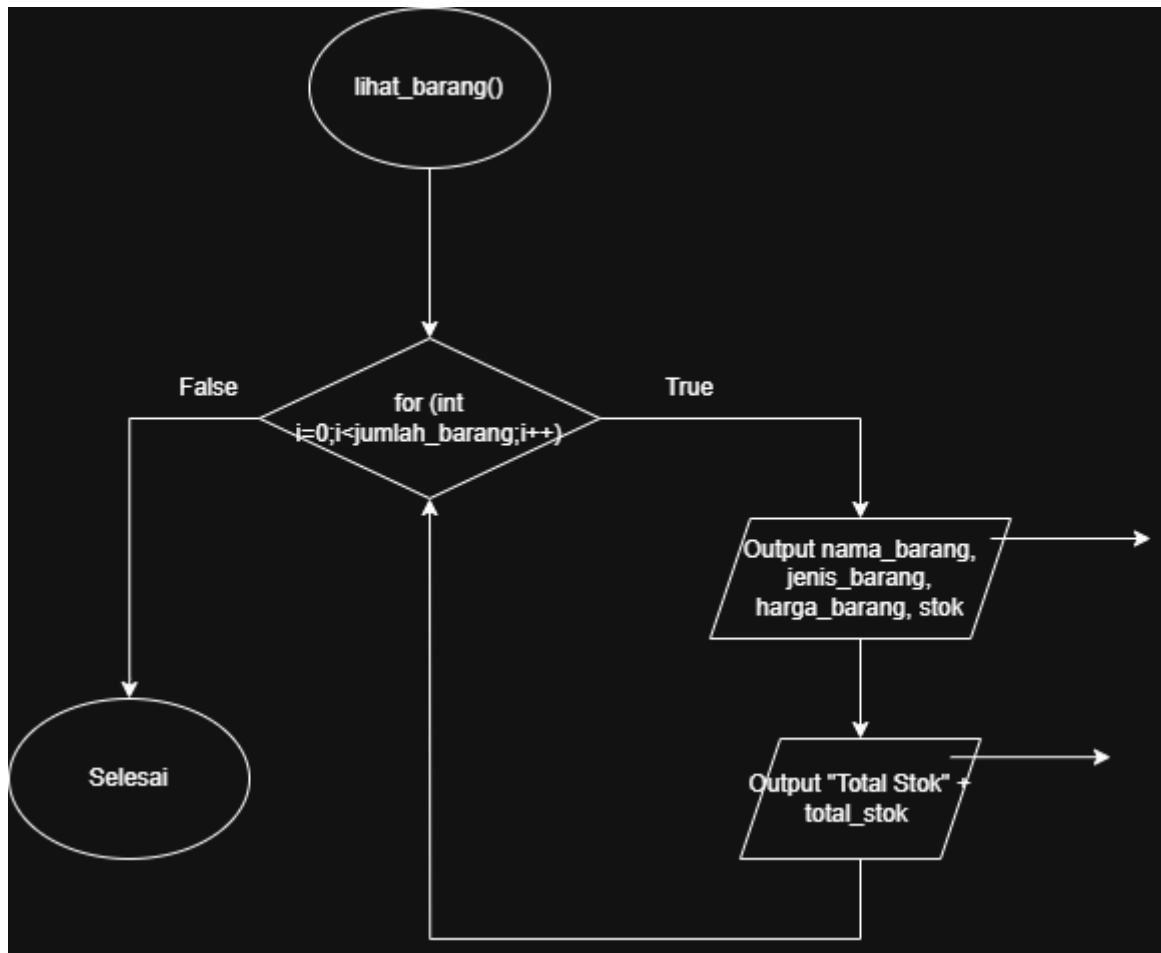
Gambar 1.4 Flowchart



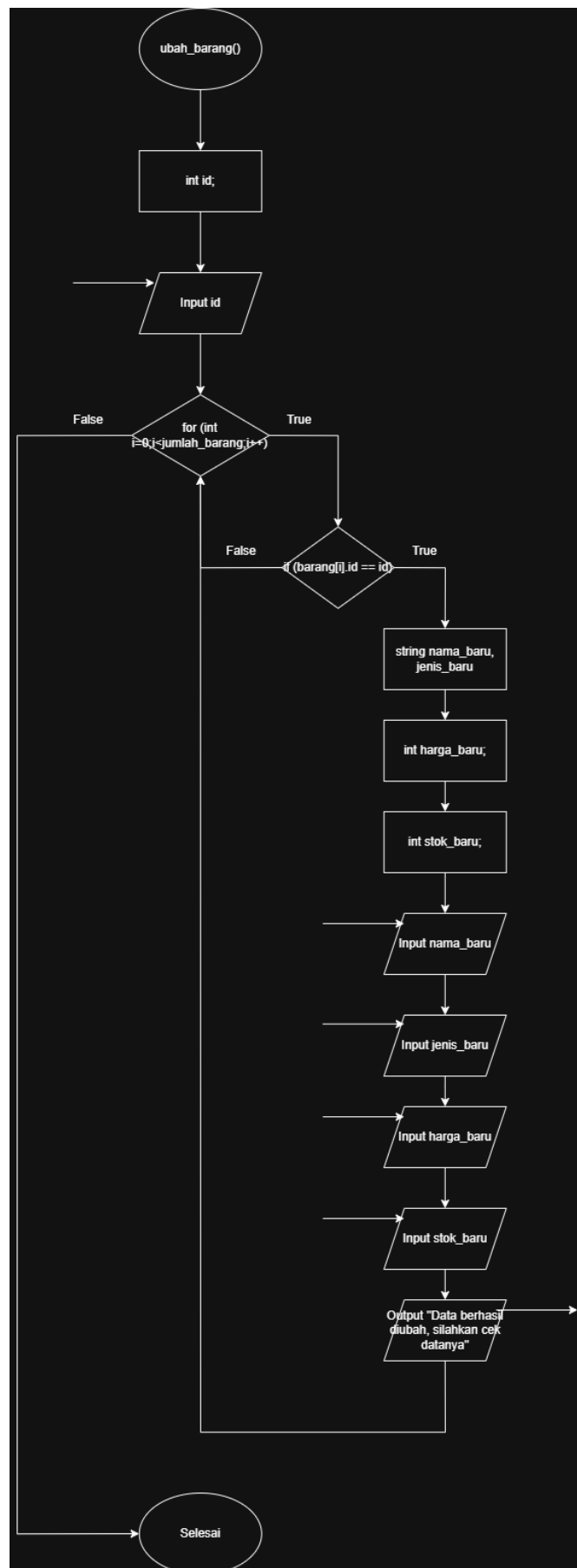
Gambar 1.5 Flowchart



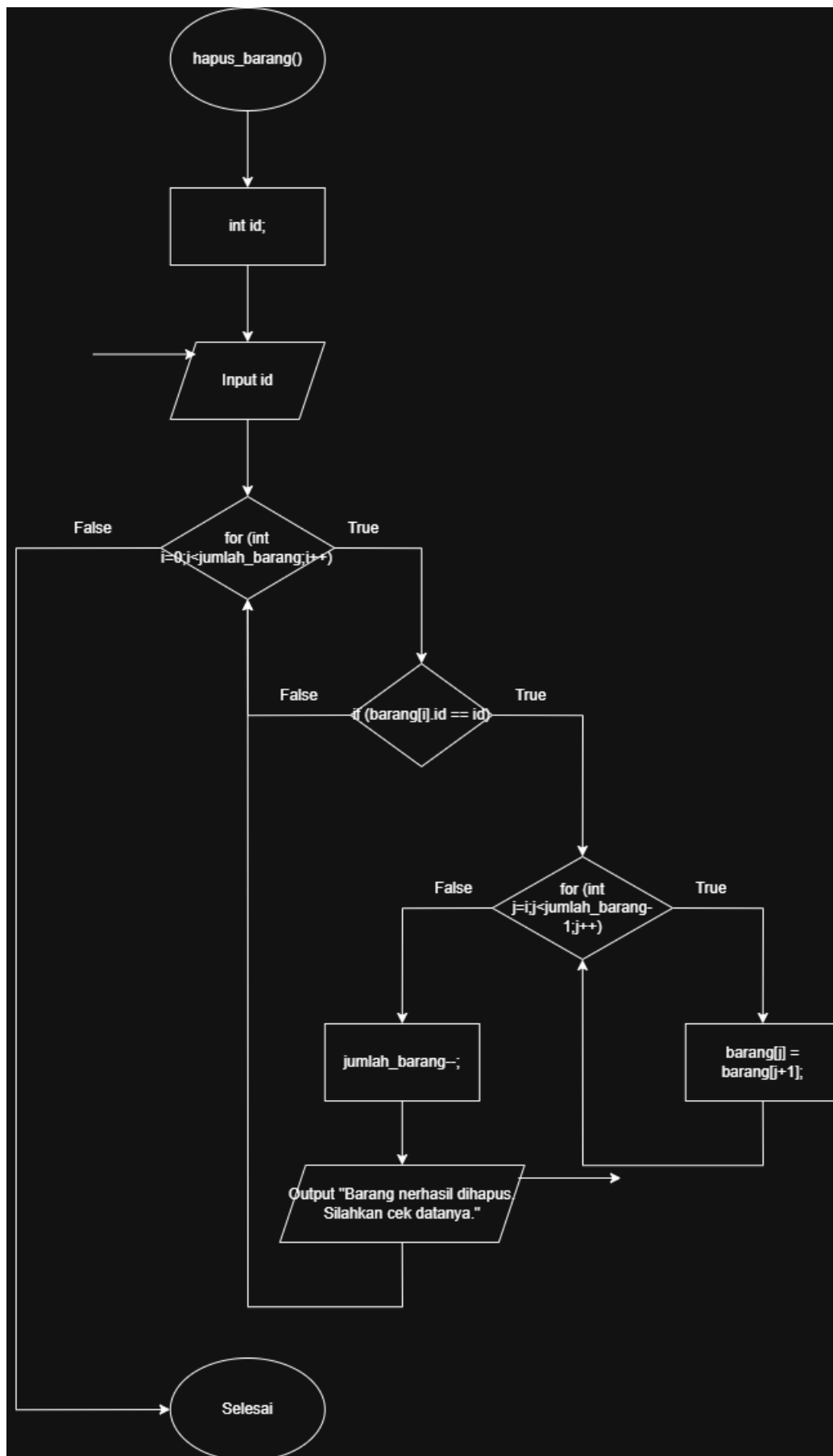
Gambar 1.6 Flowchart



Gambar 1.7 Flowchart



Gambar 1.8 Flowchart



Gambar 1.9 Flowchart

Penjelasan Flowchart :

1. Pengguna pertama kali menginputkan Pilih, input Register, Login, atau Keluar
2. Karena data User nya belum ada, maka pengguna disuruh menginputkan register terlebih dahulu
3. Di register pengguna disuruh menginputkan Username, dan password, ketika username dan password sudah bertambah, maka akan mengeluarkan output Register berhasil ditambahkan, silahkan anda login kembali
4. Maka akan kembali ke Menu Minimarket
5. Ketika pengguna menginputkan login, maka pengguna disuruh menginputkan username, dan password, dan program akan mengecek, apakah usernam dan passwordnya ada, kalau ada, maka akan ke menu utama Minimarket Muhammad Fajar
6. Kalau Username dan password tidak ada, maka program akan mengeluarkan output kesempatan login tersisa =
7. Kalau sudah lebih dari 3 kali, maka program akan kembali ke Menu Minimarket
8. Disini kalau pengguna berhasil masuk di login
9. Maka pengguna disuruh input pilih
10. Jika pengguna menginputkan Tambah Barang,
11. Maka pengguna disuruh menginputkan Nama Barang, Jenis Barang, Harga, dan Stok
12. Ketika pengguna sudah selesai menginput
13. Maka akan mengeluarkan output, Barang berhasil ditambahkan. Silahkan cek datanya
14. Maka akan kembali ke Menu Utama Minimarket
15. Ketika pengguna menginputkan Lihat Barang
16. Maka akan memunculkan Nama Barang, Jenis Barang, Harga Barang, dan Stok
17. Ketika pengguna menginputkan ubah barang,
18. Maka pengguna disuruh menginputkan Id Barang, Nama Baru, Jenis Baru, Harga Baru, dan Stok Baru
19. Ketika pengguna sudah menginputkan semuanya, maka akan mengeluarkan Output Data berhasil diubah, silahkan cek datanya.
20. Ketika pengguna memilih Hapus Barang
21. Maka pengguna akan menginputkan Id Barang
22. Ketika pengguna sudah menginputkan Id Barang,
23. Maka Data tersebut sudah terhapus
24. Dan akan kembali ke Menu Utama Minimarket
25. Ketika pengguna menginputkna 5,
26. Maka akan kembali ke Menu Minimarket
27. Ketika pengguna menginputkan 3,
28. Maka pengguna akan keluar dari program Minimarket Muhammad Fajar
29. Dan akan mengeluarkan Output Terima kasih telah menggunakan SISTEM MINIMARKET MUHAMMAD FAJAR

2. Deskripsi Singkat Program

a. Tujuan Program

Tujuan dari program ini adalah untuk membuat sistem pengelolaan data barang pada minimarket secara sederhana menggunakan bahasa pemrograman C++. Program ini bertujuan untuk membantu pengguna dalam mengelola data barang seperti menambahkan, melihat, mengubah, dan menghapus data barang sehingga pengelolaan barang menjadi lebih terstruktur dan mudah dilakukan. Selain itu, program ini juga menerapkan konsep login pengguna serta penggunaan fungsi, prosedur, rekursif, dan overloading dalam pemrograman.

b. Fungsi dan Manfaat Program

1.) Sebagai sistem pengelolaan data barang

Program ini berfungsi untuk mengelola data barang yang terdapat pada minimarket. Pengguna dapat melakukan beberapa operasi seperti:

- Menambahkan data barang
- Melihat daftar barang
- Mengubah data barang
- Menghapus data barang

Manfaatnya adalah memudahkan pengguna dalam mengatur data barang secara lebih terstruktur tanpa harus mencatat secara manual.

2.) Sebagai sistem login pengguna

Program dilengkapi dengan fitur login yang mengharuskan pengguna memasukkan username dan password yang telah didaftarkan. Pengguna diberikan maksimal tiga kali kesempatan untuk melakukan login.

Manfaatnya adalah:

- Melatih konsep keamanan dasar dalam sistem program.
- Mengajarkan penggunaan pencarian data dalam array.
- Melatih penggunaan fungsi yang mengembalikan nilai boolean.

3.) Melatih penggunaan fungsi dan prosedur

Program menggunakan beberapa fungsi dan prosedur untuk memisahkan setiap fitur program seperti proses login, registrasi pengguna, dan pengelolaan data barang.

Manfaatnya adalah:

- Membuat program menjadi lebih terstruktur dan mudah dipahami.
- Melatih penggunaan parameter dalam fungsi.
- Mempermudah pengembangan program di masa depan.

4.) Melatih penggunaan perulangan (looping)

Program menggunakan perulangan seperti for, while, dan do-while untuk menjalankan proses tertentu, seperti menampilkan menu, mencari data user, dan menampilkan data barang.

Manfaatnya adalah:

- Memungkinkan program berjalan berulang tanpa harus dijalankan kembali.
- Melatih pemahaman penggunaan perulangan dalam pemrograman.

5.) Melatih penggunaan percabangan (if-else)

Program menggunakan percabangan untuk menentukan tindakan yang akan dilakukan berdasarkan pilihan pengguna pada menu serta untuk mengecek kondisi tertentu seperti keberhasilan login.

Manfaatnya adalah:

- Membantu memahami cara program mengambil keputusan berdasarkan kondisi tertentu.
- Melatih logika pengambilan keputusan dalam pemrograman.

6.) Melatih penggunaan fungsi rekursif

Program menggunakan fungsi rekursif untuk menghitung total stok barang secara otomatis dengan memanggil fungsi itu sendiri hingga semua data stok dijumlahkan.

Manfaatnya adalah:

- Melatih pemahaman konsep rekursif dalam pemrograman.
- Menunjukkan cara alternatif dalam melakukan perhitungan berulang.

7.) Melatih penggunaan fungsi overloading

Program menggunakan fungsi overloading pada fungsi Output() yang memiliki dua bentuk fungsi dengan parameter yang berbeda.

Manfaatnya adalah:

- Memahami konsep penggunaan fungsi dengan nama yang sama tetapi memiliki parameter berbeda.
- Membuat penulisan kode menjadi lebih fleksibel dan efisien.

1.) Source Code

A. Fitur Variabel

Fitur ini berisi variabel yang akan digunakan

```
#include <iostream>
#include <iomanip>
using namespace std;

struct User{
    string username;
    string password;
};

struct Jenis_Barang{
    string kategori;
};

struct Barang{
    int id;
    string nama_barang;
    int harga_barang;
    int stok;
    Jenis_Barang jenis_barang;
};

User users[100];
Barang barang[100];

int jumlah_barang = 0;
int jumlah_user = 0;
```

B. Fitur Fungsi Rekursif

Fitur ini adalah fitur rekursif yang digunakan untuk menghitung semua stok barang.

```
int total_stok(int index){
    if(index < 0){
        return 0;
    }
    return barang[index].stok + total_stok(index-1);
}
```

C. Fitur Fungsi Overloading

Fitur ini fitur overloading yang digunakan untuk mengeluarkan output yang biasa atau yang ada nilainya (Contoh nilainya adalah output + variabel).

```
void Output(string pesan){
    cout<<pesan<<endl;
}

void Output(string pesan, int nilai){
    cout<<pesan<<nilai<<endl;
}
```

D. Fitur Fungsi Cek User

Fitur ini fitur fungsi cek user yang digunakan untuk mengecek data user nya ada atau tidak ada, kalau ada true, kalau tidak ada false.

```
bool cek_user(User users[], int jumlah_user, string username, string password){

    for(int i=0;i<jumlah_user;i++){

        if(username == users[i].username && password == users[i].password){
            return true;
        }

    }

    return false;
}
```

E. Fitur Procedure Register

Fitur procedure register ini digunakan untuk membuat akun baru.

```
void register_user(User users[], int &jumlah_user){

    cout<<"\nMENU REGISTER USER\n";

    cout<<"Username : ";
    getline(cin, users[jumlah_user].username);

    cout<<"Password : ";
    getline(cin, users[jumlah_user].password);

    jumlah_user++;

    Output("Registrasi berhasil ditambahkan, silahkan anda login kembali");
}
```

F. Fitur Procedure Tambah Barang

Fitur procedure tambah barang ini digunakan untuk menambahkan barang.

```
void tambah_barang(Barang barang[], int &jumlah_barang){

    cout<<"Silahkan Tambah Barang disini\n";

    barang[jumlah_barang].id = jumlah_barang + 1;

    cout<<"Nama Barang : ";
    cin.ignore();
    getline(cin, barang[jumlah_barang].nama_barang);

    cout<<"Jenis Barang : ";
    getline(cin, barang[jumlah_barang].jenis_barang.kategori);

    cout<<"Harga : ";
    cin>>barang[jumlah_barang].harga_barang;

    cout<<"Stok : ";
    cin>>barang[jumlah_barang].stok;

    jumlah_barang++;

    Output("Barang berhasil ditambahkan. Silahkan cek datanya");
}
```


G. Fitur Procedure Lihat Barang

Fitur procedure lihat barang ini digunakan untuk melihat keseluruhan barang apa saja yang ada, termasuk menampilkan semua stok nya.

```
void lihat_barang(Barang barang[], int jumlah_barang){

    cout<<"\nDATA BARANG\n";
    cout<<"-----\n";
    cout<<left<<setw(5)<<"ID"
        <<setw(15)<<"Nama Barang"
        <<setw(15)<<"Jenis Barang"
        <<setw(10)<<"Harga"
        <<setw(10)<<"Stok"<<endl;
    cout<<"-----\n";

    for(int i=0;i<jumlah_barang;i++){
        cout<<left<<setw(5)<<barang[i].id
            <<setw(15)<<barang[i].nama_barang
            <<setw(15)<<barang[i].jenis_barang.kategori
            <<setw(10)<<barang[i].harga_barang
            <<setw(10)<<barang[i].stok<<endl;
    }

    Output("Total stok barang : ", total_stok(jumlah_barang-1));
}
```

H. Fitur Procedure Ubah Barang

Fitur procedure ubah barang ini digunakan untuk mengubah nama barang, jenis barang, harga barang, dan stok barang. yang ingin diubah.

```
void ubah_barang(Barang barang[], int jumlah_barang){

    int id;

    cout<<"Masukkan ID barang yang ingin anda ubah : ";
    cin>>id;

    for(int i=0;i<jumlah_barang;i++){

        if(barang[i].id == id){

            string nama_baru, jenis_baru;
            int harga_baru;
            int stok_baru;

            cout<<"Nama Barang (kosongkan jika tidak ingin anda mengubah nya) : ";

            cin.ignore();
            getline(cin,nama_baru);

            if(nama_baru != ""){
                barang[i].nama_barang = nama_baru;
            }

            cout<<"Jenis Barang (kosongkan jika tidak ingin anda mengubah nya) : ";

            getline(cin,jenis_baru);

            if(jenis_baru != ""){
                barang[i].jenis_barang.kategori = jenis_baru;
            }

            cout<<"Harga (isi 0 jika tidak ingin anda mengubah nya) : ";
            cin>>harga_baru;

            if(harga_baru != 0){
                barang[i].harga_barang = harga_baru;
            }

            cout<<"Stok Baru (isi 0 jika tidak ingin anda mengubah nya) : ";
            cin>>stok_baru;

            if(stok_baru != 0){
                barang[i].stok = stok_baru;
            }
        }
    }
}
```

```

        Output("Data berhasil diubah, silahkan cek datannya");
    }

}

}

```

I. Fitur Procedure Hapus Barang

Fitur procedure hapus barang digunakan untuk menghapus barang yang tidak digunakan atau tidak perlu.

```

void hapus_barang(Barang barang[], int &jumlah_barang){

    int id;

    cout<<"Masukkan ID barang yang ingin anda hapus: ";
    cin>>id;

    for(int i=0;i<jumlah_barang;i++){

        if(barang[i].id == id){

            for(int j=i;j<jumlah_barang-1;j++){

                barang[j] = barang[j+1];

            }

            jumlah_barang--;

            Output("Barang berhasil dihapus. Silahkan cek datanya");

        }

    }

}

```

J. Fitur Procedure Menu

Fitur procedure menu digunakan untuk menampilkan semua crud(creat, read, update, and delete) barang.

```
void menu(Barang barang[], int &jumlah_barang){  
  
    int pilih;  
  
    do{  
  
        cout<<"\nSELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR\n";  
        cout<<"1. Menu Tambah Barang\n";  
        cout<<"2. Menu Lihat Barang\n";  
        cout<<"3. Menu Ubah Barang\n";  
        cout<<"4. Menu Hapus Barang\n";  
        cout<<"5. Keluar\n";  
  
        cout<<"Silahkan pilih menu : ";  
        cin>>pilih;  
  
        if(pilih == 1){  
            tambah_barang(barang,jumlah_barang);  
        }  
        else if(pilih == 2){  
            lihat_barang(barang,jumlah_barang);  
        }  
        else if(pilih == 3){  
            ubah_barang(barang,jumlah_barang);  
        }  
        else if(pilih == 4){  
            hapus_barang(barang,jumlah_barang);  
        }  
        else if(pilih == 5){  
            cout<<"Anda Keluar dari MENU MINIMARKET MUHAMMAD FAJAR\n";  
        }  
        else{  
            cout<<"Menu yang anda pilih tidak tersedia! Silahkan pilih menu 1  
sampai 5\n";  
        }  
  
    }while(pilih != 5);  
  
}
```

K. Fitur Procedure Login

Fitur procedure login digunakan pengguna untuk login/masuk ke menu crud barang.

```
void login(User users[], int jumlah_user, Barang barang[], int &jumlah_barang){

    int jumlah_login = 3;
    string username,password;
    bool status_login = false;

    cout<<"\nMENU LOGIN MINIMARKET MUHAMMAD FAJAR\n";

    while (jumlah_login > 0 && status_login == false) {

        cout<<"Username : ";
        getline(cin,username);

        cout<<"Password : ";
        getline(cin,password);

        if(cek_user(users,jumlah_user,username,password)){

            cout<<"Login berhasil, selamat datang mahasiswa\n";

            status_login = true;

            menu(barang,jumlah_barang);
            return;

        }

        jumlah_login--;

        Output("Username atau password salah, kesempatan login tersisa : ",
jumlah_login);
    }

    if (status_login == false) {
        Output("Username dan Password Anda salah sebanyak 3 kali. Kembali ke
Menu MINIMARKET MUHAMMAD FAJAR");
    }

}
```

L. Fitur Main

Fitur main adalah fitur utama dari program.

```
int main(){

    int pilih;

    do{

        cout<<"\nSELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR\n";
        cout<<"1. Register\n";
        cout<<"2. Login\n";
        cout<<"3. Keluar\n";

        cout<<"Pilih menu : ";
        cin>>pilih;
        cin.ignore();

        if(pilih == 1){
            register_user(users,jumlah_user);
        }
        else if(pilih == 2){
            login(users,jumlah_user,barang,jumlah_barang);
        }
        else if(pilih == 3){
            cout<<"Terima kasih telah menggunakan SISTEM MINIMARKET MUHAMMAD
FAJAR\n";
        }
        else{
            cout<<"Menu tidak tersedia, silahkan pilih menu 1 sampai 3\n";
        }

    }while(pilih != 3);

}
```

2.) Hasil Output

```
SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Register
2. Login
3. Keluar
Pilih menu : 1
```

Gambar 4.1 Output

```
MENU REGISTER USER
Username : Muhammad Fajar
Password : 2509106117
Registrasi berhasil ditambahkan, silahkan anda login kembali
```

Gambar 4.2 Output

```
SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Register
2. Login
3. Keluar
Pilih menu : 2
```

Gambar 4.3 Output

```
MENU LOGIN MINIMARKET MUHAMMAD FAJAR
Username : asdasd
Password : asdasd
Username atau password salah, kesempatan login tersisa : 2
Username : asdasd
Password : asdasd
Username atau password salah, kesempatan login tersisa : 1
Username : asdasd
Password : asdsad
Username atau password salah, kesempatan login tersisa : 0
Username dan Password Anda salah sebanyak 3 kali. Kembali ke Menu MINIMARKET MUHAMMAD FAJAR
```

Gambar 4.4 Output

```
SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Register
2. Login
3. Keluar
Pilih menu : 2
```

Gambar 4.5 Output

```
MENU LOGIN MINIMARKET MUHAMMAD FAJAR
Username : Muhammad Fajar
Password : 2509106117
Login berhasil, selamat datang mahasiswa
```

Gambar 4.6 Output

```
SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar
Silahkan pilih menu : 1
Silahkan Tambah Barang disini
Nama Barang : UltraMilk
Jenis Barang : Minuman
Harga : 5000
Stok : 10
Barang berhasil ditambahkan. Silahkan cek datanya
```

Gambar 4.7 Output

```
SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar
Silahkan pilih menu : 1
Silahkan Tambah Barang disini
Nama Barang : Chitatto
Jenis Barang : Snack
Harga : 5000
Stok : 10
Barang berhasil ditambahkan. Silahkan cek datanya
```

Gambar 4.8 Output

```
SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar
Silahkan pilih menu : 1
Silahkan Tambah Barang disini
Nama Barang : Aqua
Jenis Barang : Minuman
Harga : 5000
Stok : 10
Barang berhasil ditambahkan. Silahkan cek datanya
```

Gambar 4.9 Output

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR

1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar

Silahkan pilih menu : 2

DATA BARANG

ID	Nama Barang	Jenis Barang	Harga	Stok
1	UltraMilk	Minuman	5000	10
2	Chitatto	Snack	5000	10
3	Aqua	Minuman	5000	10
Total stok barang : 30				

Gambar 4.10 Output

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR

1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar

Silahkan pilih menu : 3

Masukkan ID barang yang ingin anda ubah : 3

Nama Barang (kosongkan jika tidak ingin anda mengubah nya) : Yakult

Jenis Barang (kosongkan jika tidak ingin anda mengubah nya) : Minuman

Harga (isi 0 jika tidak ingin anda mengubah nya) : 5000

Stok Baru (isi 0 jika tidak ingin anda mengubah nya) : 10

Data berhasil diubah, silahkan cek datannya

Gambar 4.11 Output

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR

1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar

Silahkan pilih menu : 2

DATA BARANG

ID	Nama Barang	Jenis Barang	Harga	Stok
1	UltraMilk	Minuman	5000	10
2	Chitatto	Snack	5000	10
3	Yakult	Minuman	5000	10
Total stok barang : 30				

Gambar 4.12 Output

```

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar
Silahkan pilih menu : 4
Masukkan ID barang yang ingin anda hapus: 3
Barang berhasil dihapus. Silahkan cek datanya

```

Gambar 4.13 Output

```

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar
Silahkan pilih menu : 2

DATA BARANG
-----
ID   Nama Barang   Jenis Barang   Harga   Stok
-----
1    UltraMilk     Minuman       5000    10
2    Chitatto     Snack         5000    10
Total stok barang : 20

```

Gambar 4.14 Output

```

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Menu Tambah Barang
2. Menu Lihat Barang
3. Menu Ubah Barang
4. Menu Hapus Barang
5. Keluar
Silahkan pilih menu : 5
Anda Keluar dari MENU MINIMARKET MUHAMMAD FAJAR

```

Gambar 4.15 Output

```

SELEMAT DATANG DI MINIMARKET MUHAMMAD FAJAR
1. Register
2. Login
3. Keluar
Pilih menu : 3
Terima kasih telah menggunakan SISTEM MINIMARKET MUHAMMAD FAJAR

```

Gambar 4.16 Output

3.) Langkah-langkah GIT

a. GIT Init

```
PS D:\Muhammad Fajar\Kuliah\Semester 2\APL\Pratikum\pratikum-apl> git init  
Initialized empty Git repository in D:/Muhammad Fajar/Kuliah/Semester 2/APL/Pratikum/pratikum-apl/.git/
```

Gambar 5.1 GIT Init

Perintah git init digunakan untuk membuat sebuah repository Git baru di dalam folder yang dipilih. Dengan kata lain, perintah ini mengubah sebuah folder biasa menjadi folder yang dapat dikelola oleh Git. Setelah menjalankan git init, Git akan membuat sebuah sub-folder tersembunyi bernama .git. Folder inilah yang menyimpan semua informasi penting terkait version control, seperti riwayat perubahan, konfigurasi, dan metadata repository.

Contohnya, ketika kita sedang membuat sebuah proyek baru, langkah pertama yang biasanya dilakukan adalah masuk ke folder proyek tersebut melalui terminal, lalu menjalankan perintah git init. Setelah itu, folder tersebut resmi menjadi sebuah repository Git sehingga setiap perubahan file yang ada di dalamnya bisa dilacak dan dikelola dengan perintah-perintah Git lainnya, seperti git add, git commit, atau git status.

Perlu diperhatikan bahwa git init hanya dijalankan sekali pada saat awal membuat repository di sebuah folder. Jika sudah menjadi repository Git, tidak perlu mengulanginya lagi.

b. GIT Add

```
PS D:\Muhammad Fajar\Kuliah\Semester 2\APL\Pratikum\pratikum-apl> git add .
```

Gambar 5.2 GIT Add

Perintah git add . digunakan untuk menambahkan semua perubahan file yang ada di dalam folder proyek ke dalam staging area Git. Staging area adalah tempat sementara di mana perubahan file disiapkan sebelum benar-benar disimpan ke dalam riwayat repository melalui perintah git commit.

Tanda titik (.) setelah git add berarti semua file yang ada di direktori saat ini, termasuk subfolder di dalamnya, akan ikut ditambahkan. Jadi, jika ada file baru, file yang sudah diubah, atau file yang dihapus, semuanya akan masuk ke dalam staging area sekaligus.

Contohnya, ketika kita selesai mengedit beberapa file sekaligus, daripada menambahkan file satu per satu dengan git add namafile, kita bisa langsung mengetik git add . agar semua perubahan tercatat dalam satu langkah. Setelah itu, barulah perubahan tersebut bisa disimpan secara permanen menggunakan git commit.

Namun, karena git add . menambahkan seluruh perubahan, kita perlu berhati-hati agar tidak secara tidak sengaja menyertakan file yang sebenarnya tidak ingin dimasukkan ke dalam repository. Untuk menghindari hal ini, biasanya digunakan file .gitignore agar Git mengabaikan file tertentu.

c. GIT Commit

```
PS D:\Muhammad Fajar\Kuliah\Semester 2\APL\Pratikum\pratikum-apl> git commit -m "Menambahkan Posttest 3"
[main eacec82] Menambahkan Posttest 3
4 files changed, 571 insertions(+)
create mode 100644 kelas/pertemuan-3/pertemuan-3.cpp
create mode 100644 kelas/pertemuan-3/pertemuan-3.exe
create mode 100644 post-test/post-test-apl-3/2509106117-Muhammad_Fajar-PT-3.cpp
create mode 100644 post-test/post-test-apl-3/2509106117-Muhammad_Fajar-PT-3.exe
```

Gambar 5.3 GIT Commit

Commit dalam Git dapat diibaratkan seperti menyimpan catatan atau rekaman atas perubahan yang telah dilakukan pada proyek. Setiap kali kita melakukan commit, Git akan menyimpan kondisi file yang sudah dimasukkan ke staging area sebagai satu titik riwayat baru. Titik riwayat ini nantinya bisa dilihat, dilacak, bahkan dikembalikan lagi jika dibutuhkan.

Sebuah commit biasanya disertai dengan pesan (commit message) yang menjelaskan apa perubahan yang dilakukan. Misalnya, ketika menambahkan fitur baru, memperbaiki bug, atau mengubah bagian tertentu dari kode. Pesan commit ini sangat penting agar orang lain — atau bahkan diri kita sendiri di kemudian hari — dapat memahami tujuan dari perubahan yang dibuat.

Setiap commit memiliki identitas unik berupa hash (serangkaian angka dan huruf) yang membuat Git bisa membedakan satu commit dengan commit lainnya. Dengan adanya commit, kita bisa menelusuri riwayat proyek dari awal hingga kondisi terbaru, serta berkolaborasi dengan lebih teratur.

d. GIT Remote

```
PS D:\Muhammad Fajar\Kuliah\Semester 2\APL\Pratikum\pratikum-apl> git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 940.37 KiB | 3.51 MiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/Celrik08/praktikum-apl.git
829ea77..eacec82  main -> main
```

Gambar 5.4 GIT Remote

Git remote adalah perintah dalam Git yang digunakan untuk menghubungkan repository lokal dengan repository yang berada di server atau layanan hosting kode, seperti GitHub, GitLab, atau Bitbucket. Dengan adanya remote, kita bisa mengirim (push) perubahan dari komputer lokal ke server, atau mengambil (pull/fetch) perubahan dari server ke komputer lokal.

Di sini, origin adalah nama default yang diberikan untuk repository remote. Setelah remote ditambahkan, kita bisa menjalankan perintah git push untuk mengunggah commit ke repository online, atau git pull untuk mengambil pembaruan dari repository tersebut. Selain itu, git remote -v dapat digunakan untuk melihat daftar remote yang sudah terhubung beserta URL-nya.

Dengan kata lain, git remote membuat proyek kita tidak hanya tersimpan di komputer lokal, tetapi juga dapat dibagikan, disinkronkan, dan dikerjakan bersama sama melalui repository yang ada di server.

e. **GIT Push**

```
PS D:\Muhammad Fajar\Kuliah\Semester 2\APL\Pratikum\pratikum-apl> git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 672.05 KiB | 7.38 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Celrik08/praktikum-apl.git
1e74212..1ed32f2 main -> main
```

Gambar 5.5 GIT Push

git push adalah perintah yang digunakan untuk mengirimkan atau mengunggah commit dari repository lokal ke repository remote, misalnya ke GitHub, GitLab, atau Bitbucket. Dengan melakukan push, semua perubahan yang sudah kita simpan melalui commit di komputer akan tersalin ke repository yang ada di server sehingga bisa diakses oleh orang lain atau digunakan pada perangkat lain.

Pada contoh di atas, origin adalah nama remote (default ketika kita menambahkan repository GitHub), dan main adalah nama branch yang akan dikirim. Artinya, semua commit yang ada di branch main pada komputer kita akan diunggah ke branch main di repository remote.

Tambahan -u (atau --set-upstream) berfungsi agar pada push berikutnya kita cukup mengetik git push tanpa harus menuliskan nama remote dan branch lagi, karena Git sudah mengingat pengaturan tersebut.

Dengan kata lain, git push adalah cara kita membagikan hasil kerja dari repository lokal ke repository online, sehingga proyek bisa ter-update dan mudah dikelola bersama tim.